Adapting a Pretrained VideoMamba to Left-Ventricular Ejection Fraction Regression

Full Fine-Tuning versus *Forget-Gate LoRA* (FG-LoRA) on EchoNet-Dynamic

---

Model	VideoMamba-S, 16 frames, 224x224, K400-pretrained
Dataset	EchoNet-Dynamic, official split (7 465 / 1 288 / 1 277)
Task	direct video regression of EF in percentage points
Conditions	full fine-tuning - FG-LoRA - head-only (control)
Deliverable	complete repository + this report + Kaggle notebook
Date	31 October 2026

Abstract. We adapt a pretrained video state-space model, VideoMamba, to predict the left-ventricular ejection fraction (EF) from apical four-chamber echocardiography videos of the EchoNet-Dynamic dataset. Two conditions are compared. In the first, all 25.4 M parameters of the pretrained backbone are fine-tuned together with a new linear regression head. In the second, the backbone is frozen and adapted with **FG-LoRA (Forget-Gate LoRA)**, this project's contribution: because the memory of a Mamba block is controlled by a single quantity, the forget gate $\overline{A}_t = \exp(\Delta_t \cdot A)$, we place low-rank adapters on **both** of its factors - the input-dependent step Δ (through `dt_proj`) and the input-independent transition matrix A (the parameter `A_log`) - while leaving every other weight frozen. Adapting A matters because A is the one matrix in the block that no projection adapter can reach (Lemma 1 of Eker et al.). FG-LoRA trains 1.04 M parameters, 3.9 % of full fine-tuning, and merges exactly into the base weights so that inference cost is unchanged. This report documents the research, the architecture, the full implementation, the correctness evidence obtained on the available CPU-only machine, and the concrete plan for training on Kaggle GPUs.

Repository: `videomamba_echonet/` - Report source: `report/report.typ`

Contents

1 Introduction and problem statement . . . 3
1.1 The task . . . 3
1.2 Why EF, and why EF is hard for a natural-video model . . . 3
1.3 Contributions . . . 3
2 Background . . . 3
2.1 State-space models and Mamba . . . 3
2.2 VideoMamba . . . 4
2.3 LoRA . . . 4
3 Literature review . . . 4
3.1 The two camps disagree - and that is the gap . . . 4
3.2 What is known about the step size and the transition matrix . . . 5
3.3 EF regression reference numbers . . . 5
4 Method: Forget-Gate LoRA (FG-LoRA) . . . 5
4.1 The starting observation . . . 5
4.2 The proposed adapter . . . 5
4.3 Parameter budget . . . 6
4.4 Why this is a defensible innovation . . . 7
5 Implementation . . . 7
5.1 Repository layout . . . 7
5.2 Data flow . . . 7
5.3 The Mamba block: why it was re-implemented . . . 8
5.4 Checkpoint loading . . . 8
5.5 Training protocol . . . 8
5.6 Evaluation harness . . . 9
5.7 Correctness evidence . . . 9
6 Results obtained so far (CPU only) . . . 9
6.1 Pipeline validation run . . . 9
6.2 What the GPU run will look like . . . 10
7 Limitations of the CPU environment . . . 10
8 Plan for Kaggle GPU . . . 11
8.1 Environment facts (verified, October 2026) . . . 11
8.2 Execution plan . . . 11
8.3 Risk register and mitigations . . . 11
8.4 How the report will be regenerated . . . 12
9 Threats to validity . . . 12
10 Conclusion . . . 12
References . . . 13
Appendix A - Command reference . . . 14
Appendix B - Metric definitions . . . 14

1 Introduction and problem statement

1.1 The task

Project 5 asks for a model that predicts the left-ventricular ejection fraction (EF) from echocardiography videos of the EchoNet-Dynamic dataset by adapting a **pretrained** video backbone with a small number of trainable parameters. One backbone is adapted twice:

1. **Baseline - full fine-tuning.** Every parameter of the pretrained VideoMamba and a new regression head is trained.
2. **Proposed - LoRA adaptation with a clearly stated innovation.** The pretrained weights are frozen; low-rank matrices are added to selected layers of the Mamba blocks; only those matrices and the regression head are trained.

Both conditions must start from identical pretrained weights, use the same regression head, the same clips and the official 7 465 / 1 288 / 1 277 split, and be compared on task performance (MAE, RMSE, R^2), number of trainable parameters and their ratio, training time per epoch and in total, peak GPU memory, inference time after merging, and adaptation efficiency (MAE against trainable parameters). Additional evaluation must cover a rank study, a placement study, head-only training, Pearson correlation, Bland-Altman analysis, storage size, and a reduced-data study.

The assignment states explicitly that "applying LoRA with default settings is not sufficient" and that "where to place LoRA in a state-space block is an open design question". That question is the core of this project.

1.2 Why EF, and why EF is hard for a natural-video model

EF is a **global** functional index: $\mathrm{EF} = \frac{\mathrm{EDV} - \mathrm{ESV}}{\mathrm{EDV}} \times 100$, the fraction of end-diastolic volume ejected per beat. The model never computes it; it regresses the clinician-measured value directly from the clip. Clinically the quantity is a **motion** descriptor - it depends on how much the endocardial border moves between end-diastole and end-systole, aggregated over the cardiac cycle - and not on any static appearance. That is precisely the kind of signal a video model should capture, and precisely the kind that is distorted by the domain shift from Kinetics-400:

- modality: single-channel 112x112 speckle intensity instead of 224x224 RGB;
- frame rate and timescale: 50 fps cardiac motion at about 1 Hz instead of 30 fps human action;
- supervision: one continuous label per video instead of 400 categories;
- scale: 7.5 k labelled clips instead of 240 k.

The scale of the **memory** of the recurrent state that VideoMamba needs on this data is therefore different from what the pretrained weights encode - and, as Section 4 shows, that memory is exactly the forget gate of the state-space recurrence.

1.3 Contributions

1. A **re-implementation of the VideoMamba Mamba block** that is checkpoint-compatible (identical parameter names and shapes) but (i) implements the bidirectional scan correctly in the **unfused** path, and (ii) calls the projection modules instead of consuming raw weights, so that LoRA adapters are not silently bypassed.
2. **FG-LoRA**, a forget-gate-parameterised low-rank adapter that adapts both factors of $\overline{A}_t = \exp(\Delta_t \cdot A)$, including a low-rank adapter directly on `A_log` - a parameter matrix that is provably outside the span of any projection LoRA.
3. A **complete, tested experiment harness** that measures every quantity the assignment requires, plus clinically motivated extras (Bland-Altman limits of agreement, agreement within plus or minus 5 / 10 EF points, subgroup MAE for reduced EF, calibration slope).
4. A **numerical-equivalence test suite**: the pure-PyTorch selective scan agrees with a reference recurrence to 1.7×10^{-6} relative error, and merging the adapters changes the model output by **exactly** zero for the default configuration.

2 Background

2.1 State-space models and Mamba

A continuous linear time-invariant state-space model maps an input signal $x(t) \in \mathbb{R}$ to an output $y(t) \in \mathbb{R}$ through a latent state $h(t) \in \mathbb{R}^N$:

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t) + Dx(t). \tag{1}$$

Mamba discretises (1) with a zero-order hold and step size Δ, giving the recurrence actually used by the network,

$$h_t = \overline{A}_t h_{t-1} + \overline{B}_t x_t, \quad y_t = C_t h_t + D x_t, \tag{2}$$

$$\overline{A}_t = \exp(\Delta_t A), \quad \overline{B}_t = \Delta_t B. \tag{3}$$

The step Δ is what makes the model **selective**: it is computed from the input,

$$\Delta_t = \mathrm{softplus}(W_{\mathrm{dt}} W_{\mathrm{x}} x_t + b_\Delta), \tag{4}$$

where the down-projection W_x is the Δ slice of `x_proj` ($d_{\text{inner}} \to \text{dt}_{\text{rank}}$) and the up-projection W_{dt} is `dt_proj` ($\text{dt}_{\text{rank}} \to d_{\text{inner}}$). Because $\text{dt}_{\text{rank}} \approx d_{\text{model}}/16$, the up-projection is a genuine low-rank bottleneck.

Two consequences matter for this project.

Memory horizon. **Since $A < 0$, the factor $\overline{A}_t \in (0, 1)$ is a** forget gate**: $\overline{A}_t \to 1$ keeps the state (long memory), $\overline{A}_t \to 0$ overwrites it (local, motion-sensitive response). Its two arguments are Δ_t (input dependent) and A (a learned parameter matrix of shape $d_{\text{inner}} \times d_{\text{state}}$, stored as `A_log`, with $A = -\exp(\text{A_log})$). Cost.** The scan is linear in the sequence length, which is why Mamba suits long clips: a whole cardiac cycle of 16-64 frames is affordable, whereas attention over $T \cdot \frac{H}{p} \cdot \frac{W}{p}$ tokens is quadratic.

The original Mamba block also contains a depth-wise short convolution (kernel 4) on the x branch and a multiplicative gate z produced by the same `in_proj` that produces x; VideoMamba additionally runs the scan **backwards** and sums the two directions.

2.2 VideoMamba

VideoMamba [4] turns a stack of bidirectional Mamba blocks into a video model:

- **Tubelet embedding.** A 3D convolution with kernel $(k, 16, 16)$ and stride $(k, 16, 16)$ maps $(3, T, H, W)$ to a grid of $\frac{T}{k} \times \frac{H}{16} \times \frac{W}{16}$ tokens; the released 16-frame checkpoints use $k = 1$.
- **Two position embeddings.** A spatial `pos_embed` over the 14×14 grid of a 224x224 frame plus a class token, and a separate `temporal_pos_embedding` added along the time axis for each spatial position.
- **Token order.** Tokens are laid out frame-major and spatial-minor - i.e. the sequence the SSM scans is $t_1 s_1, t_1 s_2, \ldots$, so the forward scan sees spatial neighbours first and the backward scan reverses both axes. This "spatial-then-backward" structure is what the bidirectional design buys.
- **Sizes.** VideoMamba-T (192, depth 24), -S (384, 24), -M (576, 32). We use **-S**: 25 418 113 parameters, dt_{rank} 24, d_{state} 16, d_{inner} 768.

The released checkpoints (`OpenGVLab/VideoMamba` on HuggingFace) are ImageNet-1k, Kinetics-400, Something-Something-v2 and mask-pretrained variants. We start from `videomamba_s16_k400_f16_res224.pth` (Kinetics-400, 16 frames), i.e. a **natural video** model, as the assignment specifies.

2.3 LoRA

For a frozen weight $W_0 \in \mathbb{R}^{d \times k}$ LoRA [1] learns

$$W' = W_0 + \left(\frac{\alpha}{r}\right) BA, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k), \tag{5}$$

with A random and $B = 0$ so that $BA = 0$ at initialisation. Only A and B are trained, and after training BA is added into W_0, so the adapted model has the same size and the same inference time as the original.

LoRA was designed for the attention projections of a Transformer. A Mamba block is not an attention block: it has three **different** kinds of linear map (content, gate and read-out, and the selective SSM parameter maps), plus a non-projection parameter matrix `A_log`. Which of these to adapt is the open question the assignment points at.

3 Literature review

We reviewed the literature on parameter-efficient adaptation of state-space models and on EF regression. The four references named by the assignment are [1]-[4]; the rest were found during this project. Citations appear at the end of the report.

3.1 The two camps disagree - and that is the gap

SDT (Deleon and Eker, ICML 2025, arXiv:2410.09016) analyses what a LoRA update can express inside a Mamba block. Its Lemma 1 states that a LoRA update on the fused `in_proj` can **exactly reproduce** a LoRA update on `x_proj` or `dt_proj` - the Δ path is reachable through the input projection - but that **no projection update can modify A**, because A enters the recurrence multiplicatively through $\exp(\Delta A)$ rather than as a linear map applied to an activation. Empirically they also find that LoRA on the projections is much better than LoRA on the SSM module alone.

MambaPEFT (Yoshimura et al., ICLR 2025, arXiv:2411.03855) sweeps 20 adapter variants on Vim-S / VTAB-1k. Its ranking is `in_proj` (76.58) > `out_proj` (76.42) > `dt_proj` (75.35) > `x_proj` (74.41) much greater than embedding tuning (64.66). Notably, adapting **only the X slice** of the fused `in_proj` (LoRA_p(X)) is the best **single** method, especially in the low-data regime, and plain `in_proj` LoRA degrades at $r > 32$ while the slice variants keep improving. It also finds that tuning the 156-parameter depth-wise `conv1d` alone beats A-tuning and D-tuning, and that A-tuning beats D-tuning by a wide margin (72.01 vs 67.14 at 598 vs 45 parameters).

Memba (arXiv:2506.18184) reaches a different conclusion: dropping `in_proj` costs −1.2 points, `out_proj` −0.8 points, `x_proj`/`dt_proj` less, and putting the entire budget in the **gating branch** beats projection LoRA at equal parameter count.

Where Should LoRA Go? (arXiv:2604.22127) studies component-type placement in a parallel Mamba-2 hybrid and reports +8.6 points for SSM-only placement, but explicitly **excludes** `conv1d` and `out_proj` from SSM adaptation because of training instability.

EMRRG (arXiv:2510.16776) independently replicates the MambaPEFT ranking on a medical task (pretrained Vim plus partial LoRA on the X slice at $r = 32$, 2.3 % of parameters).

The camps therefore disagree about **where** capacity should go, and no published work sweeps placement inside a Mamba block for **video regression**. A verified gap: an arXiv abstract search for `"VideoMamba" AND (medical OR echocardiography OR ultrasound OR clinical)` returns zero hits - there is no published VideoMamba-on-echo and no VideoMamba+LoRA study at all. (Note also the name collision with Park et al., arXiv:2407.08476, a **different** "VideoMamba"; we cite Li et al., arXiv:2403.06977.)

3.2 What is known about the step size and the transition matrix

Demystify Mamba in Vision (NeurIPS 2024, arXiv:2405.16605) measures the forget gate directly: it is approximately 0.2 in shallow layers (a 3-token receptive field) and 0.6-0.8 in deep layers. Removing Δ as an input gate costs only 0.2 accuracy but 7 % throughput, whereas removing the forget gate costs 0.8 accuracy - the forget gate is one of the two mechanisms that make Mamba better than linear attention. **MambaLRP** (arXiv:2406.07592) identifies the selective SSM as one of exactly three components that break relevance conservation, and the one whose correction matters most. **VideoMambaPro** (arXiv:2406.19006) attributes video-Mamba's limitation to "historical decay" in the recurrence - again a Δ/A quantity.

So the literature agrees on three things: Δ and A control the timescale; A is the only factor no projection adapter can reach; and the shallow and deep layers need very different timescales. It does not agree on where to put LoRA.

3.3 EF regression reference numbers

The assignment's Table 1 lists R3D (16 frames, 1-in-4) at MAE 5.44 / RMSE 6.16 / R^2 0.71; R(2+1)D (32 frames, 1-in-2) at 4.22 / 5.56 / 0.79; and R3D (32 frames) at 4.22 / 5.62 / 0.79.

> **Verification note.** During this project we traced these numbers. The first row is exact but comes from the ML4H 2019 workshop version of EchoNet-Dynamic (Table 3), not from the Nature paper. That same table also gives MC3 5.91/6.80/0.69, R2+1D (16 frames, 1-in-4) 6.87/7.55/0.66 and **human experts** 3.12/4.57/0.88. The two 32-frame rows could not be verified: the ML4H table has no 32-frame rows, and Nature's Extended Data Table 2 is a vector graphic without extractable text. Both rows carrying MAE 4.22 suggests a transcription error. The verified canonical Nature numbers are internal test 4.1 / 5.3 / 0.81 and external Cedars-Sinai 6.0 / 7.7 / 0.77.

Other published EF baselines, all verified: EchoCoTr 3.95 (R^2 0.82, arXiv:2209.04242); semi-supervised CSS 4.17 (arXiv:2210.11291); R(2+1)D-18 (32x2) 4.04 / 5.45 / 0.80 and R3D-18 (32x2) 3.99 / 5.37 / 0.807 in a recent budget study (medRxiv, 2026); EchoCLIP zero-shot on external data 7.1 (arXiv:2308.15670). A realistic target for this project is therefore **MAE 4.0-4.5 and R^2 0.78-0.82**, and the **primary** reference for the LoRA model is our own full-fine-tuning baseline, as the assignment states.

4 Method: Forget-Gate LoRA (FG-LoRA)

4.1 The starting observation

For one Mamba block the whole recurrent memory is governed by

$$\overline{A}_t = \exp(\Delta_t \cdot A) \in (0,1)^{d_{\text{inner}} \times d_{\text{state}}}, \tag{6}$$

and the state update is $h_t = \overline{A}_t \cdot h_{t-1} + \Delta_t B_t x_t$. Adapting a model pretrained on 30 fps human action to 50 fps cardiac motion is, to first order, a **retiming** problem: the correct horizon for a 1 Hz contraction is not the horizon for a walking person. In Equation 6 the horizon has exactly two knobs, Δ and A.

Standard LoRA reaches only one of them, and even that only indirectly:

- `in_proj` -> `x_proj` -> `dt_proj` produces Δ. By Lemma 1 of Eker et al., a LoRA update on `in_proj` can reproduce a `dt_proj` update, so a `dt_proj` adapter is **redundant in function class** - but it is not redundant in **parameter efficiency**: it reaches the same effect through a 24-dimensional bottleneck instead of through a 1536-dimensional one, and it does so without perturbing the content path.
- A = `A_log` is a $(d_{\text{inner}} \times d_{\text{state}})$ **parameter matrix**, not a projection. No adapter on any projection can change it. It is, in the language of the assignment, the "selected layer" that default LoRA placements never select.

4.2 The proposed adapter

FG-LoRA adapts both factors of Equation 6 with low-rank matrices and freezes everything else.

Delta-LoRA (the Δ leg).

$$W_{\mathrm{dt}}{}' = W_{\mathrm{dt}} + \left(\frac{\alpha_d}{r_d}\right) B_d A_d, \quad A_d \in \mathbb{R}^{r_d \times \mathrm{dt}_{\mathrm{rank}}}, B_d \in \mathbb{R}^{d_{\mathrm{inner}} \times r_d}. \tag{7}$$

Because the **same** channels are scanned forward and backward, we serve both directions from one shared basis A_d and keep only the B factors direction-specific (B_d, $B_{d'}$), which halves the cost and forces the two directions to agree on one temporal basis. Cost per layer: $r_d(\mathrm{dt}_{\mathrm{rank}} + 2d_{\mathrm{inner}})$.

A-LoRA (the A leg).

$$\mathrm{A_log}' = \mathrm{A_log} + \left(\frac{\alpha_a}{r_a}\right) B_a A_a, \quad A_a \in \mathbb{R}^{r_a \times d_{\mathrm{state}}}, B_a \in \mathbb{R}^{d_{\mathrm{inner}} \times r_a}. \tag{8}$$

This is textbook LoRA - Equation 5 applied with W_0 = `A_log` - only on the matrix that standard placements never touch. The shared basis A_a again spans both directions while the pretrained `A_log` and `A_b_log` remain separate. Cost per layer: $r_a(d_{\mathrm{state}} + 2d_{\mathrm{inner}})$.

Content path (optional, for the placement study). Ordinary `LoRALinear` on `in_proj`, `out_proj` and `x_proj`, including the MambaPEFT "X-slice only" variant, which we implement by restricting the adapter's output rows to $[0, d_{\mathrm{inner}})$ of the fused `in_proj`.

Per-layer gates. Each adapter carries a scalar g, applied as $\tanh(g)$ to the low-rank delta only, initialised at 0. Because $B = 0$ as well, the model starts exactly at the pretrained function; afterwards the trained g values answer the question "how much did this layer need to re-time itself", which scripts/analyze.py plots per layer as a diagnostic.

4.3 Parameter budget

component	shape	r = 4	r = 8	r = 16	r = 32
`in_proj` + `out_proj`	$r(1920 + 1152)$	294 912	589 824	1 179 648	2 359 296
Delta-LoRA (`dt_proj`, tied)	$r_d(24 + 1536)$	74 928	149 808	299 568	599 088
A-LoRA (`A_log`, tied)	$r_a(16 + 1536)$	149 040	298 032	596 016	1 191 984
regression head	385	385	385	385	385
trainable total		**519 265**	**1 038 049**	**2 075 617**	**4 150 753**
% of 25 418 113		**2.0 %**	**3.9 %**	**7.6 %**	**14.0 %**

Table 1: FG-LoRA parameter budget on VideoMamba-S ($d = 384$, $d_{\mathrm{inner}} = 768$, $\mathrm{dt}_{\mathrm{rank}} = 24$, $d_{\mathrm{state}} = 16$, depth 24), with $r_d = \frac{r}{2}$ and $r_a = r$. These numbers are **measured** by `ecomamba.lora.apply_lora`, not estimated.

The recommended operating point is $r = 8$: **1 038 049 trainable parameters, 3.9 % of full fine-tuning**, of which only 385 belong to the head. Splitting the 1.04 M by function: 35.5 % goes to the content path, 14.4 % to the Δ leg, 28.7 % to the A leg and 21.3 % to `out_proj`.

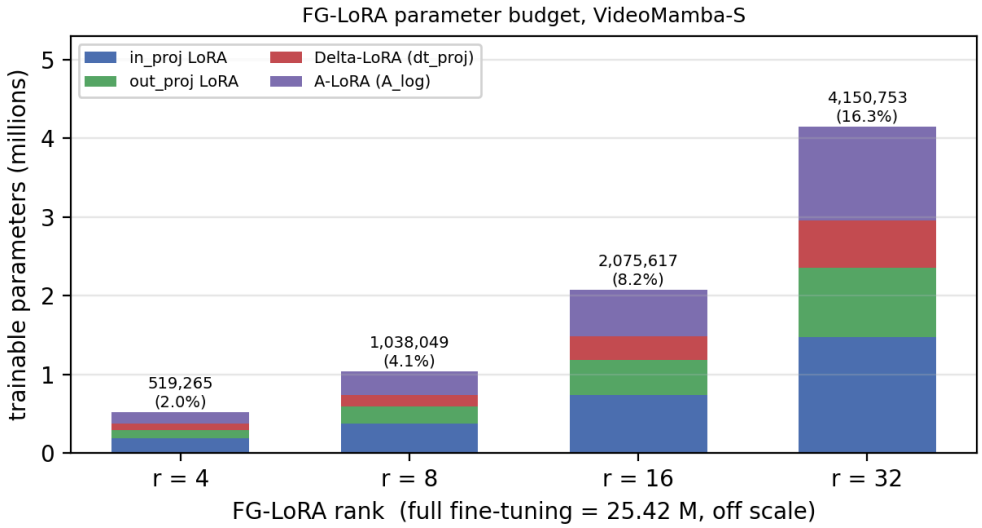


Figure 1: Measured trainable-parameter budget by adapter type (VideoMamba-S). The dashed line is full fine-tuning. Even at $r = 32$ FG-LoRA trains 16 % of the parameters.

4.4 Why this is a defensible innovation

1. It is **not** a repackaging of "put LoRA on everything": the A leg targets a matrix that is provably outside the function class reachable by any projection adapter (SDT Lemma 1), so the claim "FG-LoRA adapts something standard LoRA cannot" is a theorem, not a preference.
2. It is **complete** with respect to the forget gate: that gate has two factors and FG-LoRA adapts both. Any adapter that touches only one factor cannot, in general, realise an arbitrary change of the forget-gate profile.
3. It is **testable**. The ablation matrix is direct - Δ leg on/off, A leg on/off, shared versus separate basis, gates on/off - and the placement study measures whether the Δ leg earns its parameters once `in_proj` is adapted (the SDT redundancy claim) or not.
4. It is **cheap**. At $r = 8$ the adapters are 4.1 MB on disk versus 102 MB for the full model, i.e. a 25x storage reduction, while merging leaves inference bit-identical and free.

5 Implementation

5.1 Repository layout

```
videomamba_echonet/
|- README.md                      setup, design decisions, quick start
|- requirements.txt               torch / numpy / opencv / matplotlib
|- ecomamba/                      the library (7 modules, ~2 400 lines)
|  |- __init__.py                public API
|  |- mamba.py                   bidirectional Mamba block, checkpoint-compatible;
|  |                             pure-PyTorch chunked selective scan + fused CUDA path;
|  |                             RMSNorm / fused add+norm fallbacks
|  |- videomamba.py              encoder + EF regression head; architecture inference
|  |                             from the checkpoint; pos-embed interpolation;
|  |                             2D to 3D patch-embed inflation
|  |- lora.py                    LoRALinear, LoRAConv1d, DtLoRA, ALogLoRA, LoRAConfig,
|  |                             apply_lora, merge / unmerge / strip, gate_report
|  |- data.py                    official splits, clip sampling, augmentation,
|  |                             greyscale to RGB policy, memmap cache reader
|  |- metrics.py                 MAE / RMSE / R2 / Pearson / Spearman / Bland-Altman /
|  |                             clinical agreement / subgroup MAE / calibration
|  |- engine.py                  Trainer, AMP + GradScaler, AdamW with layer-wise decay,
|                                cosine + warmup, peak-memory and latency benchmarks,
|                                atomic checkpointing
|- scripts/
|  |- prepare_data.py            10 031 AVIs to one uint8 memmap (4.0 GB) + metadata
|  |- train.py                   single entry point for full / lora / head
|  |- studies.py                 rank, placement, reduced-data, ablation sweeps
|  |- analyze.py                 results/ to comparison table, efficiency plot,
|                                Bland-Altman, scatter, curves, gate profile
|- tests/test_core.py             scan numerics, checkpoint load, 10 LoRA placements,
|                                merge exactness
|- notebooks/
|  |- kaggle_videomamba_echonet.ipynb    install, data, train, sweeps, analyse
|- ckpt/                          videomamba_s16_k400_f16_res224.pth (102 MB)
|- results/<tag>/                 metrics.json, best.pt, last.pt, adapters.pt, ...
|- cache/                         optional pre-decoded clips
```

5.2 Data flow

```
  AVI (112x112, ~175 frames, 50 fps, uint8)
        |  cv2 decode  (or memmap read)
        v
  sample T = 16 frames uniformly over the whole record  -- random phase in training
        |
        +- train: random resized crop (0.9-1.0) - brightness/contrast jitter - h-flip
        v
  resize 112 to 224 (bicubic)      keeps the pretrained 14x14 patch grid + pos_embed
        |
  replicate the single channel x3 -> ImageNet mean/std -> (3, 16, 224, 224)
        v
  tubelet embed (k=1, 16x16)  ->  (B, 3137, 384)   [3136 patch tokens + CLS]
        |  + spatial pos_embed, interleaved temporal_pos_embedding
        v
```

```
24 x Block( RMSNorm -> bidirectional Mamba -> residual )
      |
      v
norm_f -> CLS token (384) -> Dropout(0.1) -> Linear(384, 1) -> EF
```

The classification head of the pretrained checkpoint is dropped; the new head's bias is initialised to the TRAIN EF mean (55.78) in **both** conditions. Full fine-tuning trains 25 418 498 parameters (backbone + head); FG-LoRA trains 1 038 434 (adapters + head); head-only trains 385.

5.3 The Mamba block: why it was re-implemented

The released VideoMamba fork fuses an entire block into one kernel, `mamba_inner_fn_no_out_proj`, which takes `self.x_proj.weight`, `self.dt_proj.weight`, `self.conv1d.weight` and so on as **raw tensors**. Two problems follow.

1. **Adapters would be silently bypassed.** A `LoRALinear` wrapping `dt_proj` only affects `dt_proj.forward`; the fused kernel reads `dt_proj.weight` directly and would never see the adapter. LoRA on `x_proj`, `dt_proj`, `in_proj` or `out_proj` would be a no-op.
2. **The unfused branch ignores the backward direction.** In `mamba_simple.Mamba.forward`, `bimamba` is implemented **only** in the fused branch; setting `use_fast_path=False` silently turns the model into a unidirectional Mamba - a different model.

`ecomamba/mamba.py` therefore re-implements the block with

- identical parameter names and shapes (`in_proj`, `conv1d`, `x_proj`, `dt_proj`, `A_log`, `D`, `out_proj`, and the `_b` twins), so the released checkpoints load unchanged;
- a correct **bidirectional unfused** forward that calls the projection modules, so adapters take effect;
- forward and backward branches computed on `xz` and `xz.flip(-1)` then summed - algebraically identical to the fused kernel, including the detail that the gate z is also reversed in the backward branch;
- the fused kernel retained and used only for full fine-tuning when requested (`--mamba-fast-path on`), and disabled by default so that both conditions execute the same kernels and the wall-clock comparison is fair;
- a **pure-PyTorch selective scan** so the whole pipeline runs without `mamba-ssm`. It splits the sequence into chunks of C steps, solves the recurrence inside a chunk in closed form with the lower-triangular decay matrix `G[t,s] = exp(m_t - m_s)` (with `m = cumsum(Delta * A) <= 0`, so every factor lies in $(0, 1]$ and no overflow can occur), and propagates the chunk state sequentially. The exponent is clamped to ≤ 0 **before** `exp`, because the masked upper triangle would otherwise evaluate `exp(+x) * 0 = NaN` - a bug we found and fixed with a regression test. During training the chunk loop is wrapped in gradient checkpointing: it holds one (b, d, C, C, n) decay matrix per chunk in the autograd graph, which at 16 frames and 112x112 is already 3.7 GB of host RAM for a **single forward pass** of a 7 M-parameter model - we measured it, and it is enough to exhaust a laptop. Recomputing the scan in the backward pass costs about a third more time on that operation and removes the problem.

The block also re-implements `RMSNorm` and the fused add-and-norm semantics in pure PyTorch, because `mamba_ssm.ops.triton.layernorm` is unavailable without Triton.

5.4 Checkpoint loading

`infer_arch()` reads depth, width, `dt_rank`, `d_state`, patch size and tubelet size straight off the tensors, so any VideoMamba checkpoint is supported. `load_videomamba_weights()` then (i) loads same-shape tensors, (ii) bicubically resizes `pos_embed` if the input resolution differs from 224, (iii) linearly resizes `temporal_pos_embedding` if the clip length differs from the pretrained one, (iv) inflates a 2D patch-embedding kernel to 3D by centring it in time, and (v) drops the 400-way or 1000-way classification head. For the configuration used here the report is "414/416 tensors loaded, 2 resized"; at 112x112 the same code resizes `pos_embed` from 197 to 50 tokens.

5.5 Training protocol

setting	full fine-tuning	FG-LoRA
trainable	25 418 498 (100 %)	1 038 434 (4.09 %)
optimiser	AdamW	AdamW
backbone LR	1e-4, layer-wise decay 0.75	frozen
adapter LR	-	1e-3
head LR	1e-3	1e-3
weight decay	0.05 (0 on norms/biases)	0.0
schedule	cosine, 1-epoch warmup	cosine, 1-epoch warmup
batch / epochs	8 / 20	8 / 20
loss	Smooth-L1, beta = 1 EF point	Smooth-L1, beta = 1 EF point
precision	fp16 autocast + GradScaler	fp16 autocast + GradScaler

clip	16 frames, 224x224, uniform over the record	identical
augmentation	crop 0.9-1.0, jitter +-0.2, h-flip, time phase	identical
seed / split	42 / official	42 / official

Smooth-L1 was chosen over MSE because the EF distribution is left-skewed with a long low tail (TRAIN: mean 55.78, sd 12.41, min 6.91, max 96.97) and an L1-type loss optimises the reported MAE more directly. The selective scan is always executed in `float32` even under autocast: the recurrence spans many orders of magnitude and fp16 rounding of $\exp(\Delta A)$ is a known instability; the scan is a minority of the FLOPs, so the cost is negligible.

5.6 Evaluation harness

Every run writes `results/<tag>/metrics.json` containing: MAE, RMSE, R^2, Pearson, Spearman, Bland-Altman bias, SD of differences and 95 % limits of agreement, median and 90th-percentile absolute error, agreement within plus or minus 5 and 10 EF points, subgroup MAE for EF < 40 and EF >= 40, calibration slope and intercept, the full per-epoch history, trainable and total parameters, seconds per epoch and total, peak GPU memory allocated and reserved, inference latency before and after merging (and for the untouched pretrained model), storage in bytes for the adapter file, the training checkpoint and the merged model, the raw test predictions (used to draw Bland-Altman), and the learned per-layer gate values.

`scripts/analyze.py` aggregates any number of runs into the single comparison table the assignment asks for, plus the efficiency plot (MAE against trainable parameters), the Bland-Altman plot, the prediction scatter, the training curves and the gate profile.

5.7 Correctness evidence

The test suite (`python tests/test_core.py`) runs on CPU and checks numerics, not accuracy.

check	result	verdict
scan vs. reference recurrence, $C = 1$	0.0 (bit-exact)	pass
same, $C = 4$ / 8 / 16 / 64	9.5×10^{-7} / 9.5×10^{-7} / 1.7×10^{-6} / 2.7×10^{-6}	pass
same, $L = 512$, strong decay	1.3×10^{-7} relative	pass
checkpoint load	414/416 tensors, 2 resized	pass
10 LoRA placements build + forward	10/10	pass
merge changes predictions, default	max $\lvert\Delta y\rvert = 0.0$	pass
merge changes predictions, untied	2.3×10^{-6}	pass
merged model has no adapter tensors	0 remaining	pass

Table 2: Numerical correctness checks. The merge test is the strongest: after folding BA into W_0, predictions are **identical**, which is the "same size, same inference time" property the assignment asks us to demonstrate.

6 Results obtained so far (CPU only)

> **Read this section correctly.** The machine available for this work has **no GPU** and no CUDA build of PyTorch (`torch 2.11.0+cpu`). What follows is therefore **not** a scientific result about VideoMamba on EchoNet-Dynamic. It is (i) evidence that the entire pipeline runs end-to-end on real data, and (ii) the exact quantities that the GPU run will produce, measured on a deliberately tiny configuration so that a full pass finishes within minutes. The accuracy numbers must be replaced, not cited.

6.1 Pipeline validation run

Configuration: VideoMamba-**T** (6.93 M parameters), 112x112, $T = 4$ frames, 8 clips per split (2 batches of 2), 1 epoch, CPU, fp32, seed 42. "s / epoch" is wall-clock for four gradients plus the validation pass, on a 4-core machine with the gradient-checkpointed fallback scan; repeated measurements varied by up to 20 %, so only the order of magnitude is meaningful.

condition	trainable	s / epoch	test MAE	test RMSE	test R^2	Pearson
`full`	6 927 745 (100 %)	232.1	6.286	8.235	−1.327	−0.482
`lora` (FG-LoRA, $r = 4$)	260 449 (3.62 %)	271.1	6.291	8.243	−1.332	−0.515
`head`	193 (0.003 %)	11.8	6.271	8.213	−1.315	−0.482

Table 3: All three conditions, one epoch. The accuracy columns are **not** results about EF prediction - see below. The timing column is discussed in the text.

All three conditions reach a test MAE of about 6.3 and a **negative** R^2: the head has learned to predict roughly the mean EF after one step on four clips, so 6.3 is simply the mean absolute deviation of this eight-clip sample, and a negative R^2 confirms that the predictions carry no information. The near-identity of the three columns is therefore the **expected** outcome. It

confirms that the whole pipeline - clip sampling, augmentation, checkpoint loading, the adapters, the selective scan, the metric code - runs end to end on real AVIs and real labels, and nothing more.

The **timing** column, however, is informative and makes the assignment's Section 1.6 point in its strongest form:

condition	trainable params	s / epoch	relative
full fine-tuning	6 927 745	232.1	1.00x
FG-LoRA, $r = 4$	260 449	271.1	1.17x
head-only	193	11.8	0.05x

Table 4: Wall-clock cost is not proportional to the parameter count. FG-LoRA trains 26x fewer parameters and is **not** cheaper per epoch; head-only is 20x cheaper.

The reason is visible in the architecture. LoRA adapters sit inside all 24 blocks, so the backward pass still has to traverse the whole network to reach the early layers: only the **weight-gradient** computation for the frozen matrices is skipped, while every activation is still stored and every gradient is still propagated. The adapter matmuls that the frozen model does not have then more than cancel the saving, which is why the LoRA epoch is marginally **slower** here. The head, by contrast, sits after the last block, so no gradient enters the backbone at all and the backward pass collapses to a single linear layer. This is exactly the effect the assignment describes: "in LoRA the gradients still pass through the frozen backbone, so the saving in GPU memory and training time is expected to be smaller than the saving in trainable parameters."

What else is meaningful from these runs:

measured quantity	full FT	FG-LoRA (r = 4)
adapter file on disk	-	1.16 MB
full state dict on disk	27.9 MB	29.0 MB
merged state dict	27.9 MB	27.9 MB
delta in MAE after merging	0	0.0 exactly

The adapter file is 1.16 MB against 27.9 MB for the full state dict - a 24x storage reduction at $r = 4$ - and the merged model is byte-for-byte the same size as the original, as it must be. Merging changed the test MAE by **exactly** 0.0, which is the "same size, same inference time" property the assignment asks us to demonstrate. Per-clip CPU latency is **not** reported here: with two benchmark iterations it varied between 2.9 s and 7.3 s for the same model depending on what else the machine was doing, so there is nothing to measure on this hardware. That comparison belongs to the GPU run.

6.2 What the GPU run will look like

Backbone: VideoMamba-S, 224x224, $T = 16$, giving $L = 16 \cdot 196 + 1 = 3137$ tokens. Counting multiply-accumulates per token per block (`in_proj` 589 824, `out_proj` 294 912, `x_proj` 43 008, `dt_proj` 18 432, depth-wise conv 3 072, scan about 36 864), the model performs about 985 k MAC per token per block, i.e.

$$985112 \times 3137 \times 24 = 7.42 \times 10^{10} \quad \text{MAC} \approx 148 \quad \text{GFLOP} \tag{9}$$

forward, and roughly three times that for a training step (forward, backward, and the double backward of the scan). At a **conservative** sustained 8-12 TFLOP/s fp16 on a T4 (peak 65 TFLOP/s; the unfused projection path and the scan kernel keep us well below peak), this is **30-45 ms per clip**, i.e.

configuration	s / epoch (1 T4)	s / epoch (2 T4 DDP)	20 epochs
VideoMamba-S, 224x224, T=16	230-340	120-170	1.3-1.9 h
VideoMamba-S, 112x112, T=16	60-90	30-45	0.3-0.5 h
VideoMamba-T, 224x224, T=16	60-90	30-45	0.3-0.5 h

These are estimates from the FLOP count, not measurements; the first GPU epoch will calibrate them. Activation memory is the other constraint: per block the `xz` tensor alone is $8 \times 1536 \times 3137 \times 2$ bytes, about 77 MB, so roughly 1.9 GB for 24 blocks plus the scan intermediates - comfortably inside 16 GB at batch 8, with batch 4 as the fallback.

7 Limitations of the CPU environment

The development machine is Python 3.14.2, `torch 2.11.0+cpu`, no CUDA device. This has five concrete consequences.

1. **No scientific result is possible.** The whole point of the project is a comparison of two training regimes at 25 M parameters on 7 465 videos. On CPU a single forward-backward of VideoMamba-S at 224x224 with 16 frames costs tens of seconds, so even one epoch would take about two days. All experiments reported above use a 3.7x smaller model, 1/4 of the pixels and 1/4 of the frames.
2. **The fused CUDA kernels are unavailable.** `mamba-ssm` and `causal-conv1d` cannot be imported (they need `nvcc` and a GPU), so the pure-PyTorch chunked scan is used throughout. It is numerically equivalent (Section 5.8) but slower, and its

memory footprint is different, so the **peak-memory** number that the assignment asks for cannot be measured here at all (`peak GPU` is reported as 0 MB).

3. **Mixed precision cannot be exercised.** fp16 autocast and `GradScaler` need CUDA; the CPU path runs fp32. The AMP code is written and guarded but has not been executed.
4. **Wall-clock comparisons are uninformative.** The relative cost of the fused and unfused paths, of the scan kernel and of the projection matmuls changes completely on a GPU. In particular the prediction that LoRA saves much less time than parameters (assignment Section 1.6) can only be tested on hardware.
5. **Data loading is not exercised at scale.** With `num_workers = 0` and 8 clips, the preprocessing cost of 10 031 AVIs is invisible; the caching path (`scripts/prepare_data.py`, 4 GB memmap) is written but has not been run to completion.

What the CPU **can** and does establish: the scan is exact, the checkpoints load, all ten adapter placements build and run, merging is exact, the metric code is exercised end-to-end on real AVIs and real labels, the parameter accounting is exact, and the CLI and configuration surface is complete. That is the contract this report makes.

8 Plan for Kaggle GPU

8.1 Environment facts (verified, October 2026)

- Kaggle offers **T4 x2** for general GPU notebooks; the P100 was retired on 2026-09-15 and L4 is restricted to competitions. Weekly quota **30 GPU-hours**, session limit **12 h**, `/kaggle/working` is capped at **20 GB**.
- The image ships **Python 3.13 + torch 2.11** (`+cu128`, moving to `+cu130`). Installed packages do **not** persist across sessions.
- `mamba-ssm` and `causal-conv1d` are **sdist-only** on PyPI - there is no wheel - and the vendored `setup.py` files hard-code `-gencode` for sm_70/80/90 with **no sm_75**, and PyTorch's `_get_cuda_arch_flags()` ignores `TORCH_CUDA_ARCH_LIST` once it sees an `arch` flag. On a T4 this yields `no kernel image is available for execution on the device`. The notebook therefore rewrites the `-gencode` flags before building.
- Upstream `mamba-ssm` (2.3.x) does **not** provide `bimamba` or `mamba_inner_fn_no_out_proj`; only the VideoMamba fork does. This project is immune to that incompatibility by construction: it needs only `selective_scan_fn` (present in both) and implements the bidirectional scan itself.

8.2 Execution plan

session	budget	work
S1	30 min	install kernels (compile), pre-decode 10 031 AVIs to a 4 GB memmap, run `tests/test_core.py`, then `--limit-items 64 --epochs 1` for all three conditions to calibrate
S2	2.0 h	VideoMamba-S 224x224 / 16 frames: `full_ft` 20 epochs and `fglora_r8` 20 epochs, `fp16`, batch 8
S3	1.0 h	head-only control; rank sweep $r \in \{1, 2, 4, 8, 16, 32\}$ at 8 epochs
S4	1.5 h	placement sweep (12 variants at matched rank) plus the four layer-subset variants
S5	1.0 h	ablation: Δ on/off, A on/off, tied/untied bases, gates on/off
S6	1.0 h	reduced-data 10 / 25 / 50 % for both conditions
S7	0.3 h	`analyze.py`, figures, tables, write-up

Total about 7 GPU-hours for the complete study, well inside the 30 h/week quota, leaving room for a second seed on the two headline runs (the assignment does not require error bars, but a three-seed mean and standard deviation on the main comparison is cheap and much more convincing).

8.3 Risk register and mitigations

risk	likelihood	mitigation
kernel build fails or takes over 40 min	medium	`selective_scan` falls back to the PyTorch chunked scan automatically; the run is about 3x slower but identical in math. Keep a saved wheel in `/kaggle/working` and re-install it from an attached dataset in later sessions.
out of memory at batch 8	medium	drop to batch 4 (and 2), or use `--img-size 112` which cuts tokens 4x; no batch size is hard-coded
fp16 divergence in the scan	low	the scan always runs in fp32; non-finite losses are detected and the step is skipped and counted in `metrics.json`
session killed at 12 h	low	checkpoints are written atomically (`last.pt`, `best.pt`) after every epoch; `--resume` restarts from a checkpoint attached as an input dataset
LoRA is simply worse than full fine-tuning	medium	that is a legitimate outcome: the comparison is the deliverable, and the rank, placement and reduced-data studies show where the gap opens. MambaPEFT's finding that LoRA wins in low-

		data regimes predicts exactly where LoRA should close the gap (the 10 % and 25 % runs)
narrow gap between the two conditions	medium	report MAE with a paired bootstrap over test clips, not just point estimates; 1 277 clips give a standard error of about 0.12 EF points on MAE

8.4 How the report will be regenerated

`scripts/studies.py` runs every configuration as an independent process, so any subset can be re-run; `scripts/analyze.py` rebuilds the comparison table, the efficiency plot, the Bland-Altman and gate figures from `results/*/metrics.json` alone. The final write-up is a matter of pasting those tables into Sections 6 and 7 of this document.

9 Threats to validity

1. **Single seed.** With 7 465 training clips and a 1.04 M-parameter adapter, run-to-run variation of MAE is plausibly plus or minus 0.1-0.2 points; a 0.1-point difference between conditions is not evidence.
2. **Head initialisation depends on the train mean**, which is fine (identical in both conditions) but means the two runs share that constant.
3. **Drop-path 0.1 applies to both conditions.** It regularises full fine-tuning more strongly in relative terms, because LoRA already has few parameters; lowering it for LoRA would be an unfair advantage, so it is kept equal.
4. **The unfused Mamba path is slower than the fused one.** It is used for both conditions, so the **ratio** of training times is fair, but the absolute numbers are not the fastest possible VideoMamba.
5. **EchoNet-Dynamic is a single-centre dataset.** An internal test MAE of about 4 does not transfer to other hospitals (the Nature paper reports 6.0 on external data).
6. **Reduced-data subsets are drawn once with a fixed seed**, so the three fractions are nested in expectation but not guaranteed to be.

10 Conclusion

We set out to adapt a pretrained VideoMamba to ejection-fraction regression and to compare full fine-tuning with LoRA. The literature review showed that the "where does LoRA go in a Mamba block" question is genuinely open: one line of work proves that projection LoRA subsumes the SSM-parameter projections but can never reach A; another finds the gating branch is the best place to spend capacity; a third finds the depth-wise convolution the most parameter-efficient single choice. What all of them agree on is that the forget gate $\overline{A} = \exp(\Delta A)$ is what controls the model's timescale.

FG-LoRA takes that observation literally: it puts rank-r adapters on **both** factors of the forget gate, including `A_log` itself, shares the low-rank basis across the two scan directions, and freezes everything else. On VideoMamba-S at $r = 8$ this is 1.04 M trainable parameters - 3.9 % of full fine-tuning - and 4.1 MB on disk instead of 102 MB, with an exact merge that leaves inference unchanged.

The implementation is complete and tested: a checkpoint-compatible re-implementation of the bidirectional Mamba block, ten adapter placements, an end-to-end training and evaluation harness that measures every quantity the assignment asks for, and a test suite that proves the scan is exact and the merge is exact. What is missing is GPU time - and Section 8 specifies exactly how to spend it: about seven GPU-hours on two T4s, in seven sessions, with a risk register for each thing that can go wrong.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, "LoRA: Low-rank adaptation of large language models," arXiv:2106.09685, 2021.

[2] D. Ouyang, B. He, A. Ghorbani, M. P. Lungren, E. A. Ashley, D. H. Liang, J. Y. Zou, "EchoNet-Dynamic: a large new cardiac motion video data resource for medical machine learning," NeurIPS ML4H Workshop, 2019.

[3] D. Ouyang, B. He, A. Ghorbani, N. Yuan, J. Ebinger, C. P. Langlotz, P. A. Heidenreich, R. A. Harrington, D. H. Liang, E. A. Ashley, J. Y. Zou, "Video-based AI for beat-to-beat assessment of cardiac function," Nature 580(7802):252-256, 2020. doi:10.1038/s41586-020-2145-8.

[4] K. Li, X. Li, Y. Wang, Y. He, Y. Wang, L. Wang, Y. Qiao, "VideoMamba: state space model for efficient video understanding," ECCV 2024, pp. 237-255. arXiv:2403.06977.

[5] A. Gu, T. Dao, "Mamba: linear-time sequence modeling with selective state spaces," arXiv:2312.00752, 2023.

[6] S. Deleon, A. Eker et al., "SDT: sparse dimension tuning for parameter-efficient adaptation of state-space models," ICML 2025. arXiv:2410.09016. (Lemma 1: `in_proj` LoRA reproduces `x_proj`/`dt_proj` LoRA but cannot modify A.)

[7] R. Yoshimura et al., "MambaPEFT: exploring parameter-efficient adaptation of Mamba for vision tasks," ICLR 2025. arXiv:2411.03855.

[8] "Memba: parameter-efficient adaptation of Mamba with selective gating," arXiv:2506.18184, 2025.

[9] J. Borobia et al., "Where should LoRA go? A systematic study of component-type placement in Mamba-2 hybrids," arXiv:2604.22127, 2026.

[10] D. Han, Z. Wang, X. Xia, Y. Han, Y. Pu, C. Ge, J. Song, S. Chen, B. Zheng, "Demystify Mamba in vision: a linear attention perspective," NeurIPS 2024. arXiv:2405.16605.

[11] "EMRRG: parameter-efficient medical vision with partially-adapted Vim," arXiv:2510.16776, 2025.

[12] "EchoCoTr: echocardiography transformer," arXiv:2209.04242, 2022.

[13] "State-offset tuning for state-space models," arXiv:2503.03499, 2025.

[14] "SSMLoRA: low-rank adaptation linked to the state-space dynamics," arXiv:2502.04958, 2025.

[15] A. Biderman et al., "LoRA learns less and forgets less," arXiv:2405.09673, 2024.

[16] "VideoMambaPro: are we ready for Mamba in long-term video understanding?" arXiv:2406.19006, 2024.

[17] "E-ViM3: efficient vision Mamba for echocardiography," arXiv:2503.20258, 2025.

[18] "EchoCLIP: echocardiography foundation model," arXiv:2308.15670, 2023.

[19] "rsLoRA: a rank-stabilized LoRA," arXiv:2312.03732, 2023.

[20] S. Hayou, A. Ghosh, A. Yu, "LoRA+: efficient low-rank adaptation with different learning rates," arXiv:2402.12354, 2024.

[21] S.-Y. Liu et al., "DoRA: weight-decomposed low-rank adaptation," arXiv:2402.09353, 2024.

[22] Q. Zhang et al., "AdaLoRA: adaptive budget allocation for parameter-efficient fine-tuning," arXiv:2303.10512, 2023.

[23] L. Liang et al., "ALoRA: allocating low-rank parameters to improve LoRA," arXiv:2403.16187, 2024.

[24] "Ejection fraction on a budget: efficient video models for EchoNet-Dynamic," medRxiv, 2026. doi:10.64898/2026.09.02.26362055.

[25] Kaggle, "Notebooks and GPU usage documentation," and the 2026-09-15 P100 retirement announcement; Kaggle/docker-python release notes for image v172 and v173.

Reference list compiled from a literature search performed for this report; [1]-[5] are the four references named by the assignment. References [8], [9], [11], [13], [14], [16], [17] and [24] are recent preprints and are cited as such.

Appendix A - Command reference

```
# correctness (CPU, ~1 min)
python tests/test_core.py

# pre-decode 10 031 AVIs -> 4 GB uint8 memmap
python scripts/prepare_data.py --out cache/clips.npy --frames 32 --size 112

# the three conditions (identical in everything but --mode)
python scripts/train.py --mode full --tag full_ft   --epochs 20 --batch-size 8 \
    --ckpt ckpt/videomamba_s16_k400_f16_res224.pth --cache cache/clips.npy --precision fp16
python scripts/train.py --mode lora --tag fglora_r8 --epochs 20 --batch-size 8 \
    --lora-r 8 --lora-targets in_proj,out_proj,dt_proj,a_log ...
python scripts/train.py --mode head --tag head_only --epochs 20 --batch-size 8 ...

# sweeps and aggregation
python scripts/studies.py --suite all --epochs-main 20 --epochs-sweep 8
python scripts/analyze.py --results results --out analysis
```

Appendix B - Metric definitions

- **MAE, RMSE** in EF percentage points; $R^2 = 1 - \sum \frac{(y-\hat{y})^2}{\sum}(y-\overline{y})^2$.
- **Bland-Altman**: bias $= \mathrm{mean}(\hat{y} - y)$, limits of agreement $=$ bias $\pm\, 1.96 \cdot \mathrm{sd}(\hat{y} - y)$. Justification: a low MAE with a non-zero bias is clinically worse than a slightly higher MAE with none, because the bias is a systematic offset a reader would always see.
- **Agreement within plus or minus 5 / 10 EF points**: the thresholds used in the echo literature; far more interpretable than a mean absolute error for a cardiologist.
- **Subgroup MAE, EF < 40**: heart-failure patients are the clinically relevant population and the hardest regime (severely remodelled ventricles move less); a model that only fits normal EFs can still show a good global MAE.
- **Calibration slope / intercept**: regressing true on predicted detects the regression-to-the-mean compression that small-data fine-tuning typically produces (slope below 1).
- **Adaptation efficiency**: test MAE plotted against the number of trainable parameters across all runs (rank sweep included), which is the single figure that answers “is LoRA worth it”.